

EF01 Task - Theory Answers

Entity Framework Core (Code First) - Bookstore System

Task 3 - Understanding the Up() and Down() Methods

Every migration file generated by Add-Migration contains two methods:

- **Up()** contains the operations that move the database schema **forward** to the new version. For the InitialCreate migration, it creates the tables for the entities (with their columns, primary keys, and constraints).
- **Down()** contains the exact **reverse** of those operations (for example, dropping the tables that Up() created), so the change can be rolled back, either by updating the database to an earlier migration or by removing the migration.

EF Core records which migrations have already been applied in a table called **__EFMigrationsHistory**, and keeps a **ModelSnapshot** file that describes the current model, so the next Add-Migration only generates the differences since the last one.

Task 4, Part A - Mapping by Convention

Q1: Why did the property "Id" become a Primary Key without any explicit configuration?

Because of EF Core's built-in **primary key convention**: a property named **Id** or **<ClassName>Id** (for example, **BookId**), case-insensitive, is automatically treated as the entity's primary key. Since its type is **int**, EF Core also configures it by convention as an **identity column** (auto-generated by SQL Server), so no attribute or Fluent API is needed.

Q2: Why is "Country" nullable in the database while "Price" is not?

EF Core infers column nullability from the **C# type**. **Country** is declared as **string?** (a nullable type), so its column allows **NULL**. **Price** is a **decimal**, a non-nullable value type that can never hold null, so its column is created as **NOT NULL**. The same logic applies to **PublishedDate**, which is **DateTime?** and therefore nullable. (With nullable reference types enabled, a plain **string** would also become **NOT NULL**, while **string?** stays nullable.)

Task 5 - Why Code First and Why Fluent API?

1. Why do we use Code First rather than Database First?

With Code First, the **C# classes are the single source of truth**: the database schema is generated from the code through migrations, instead of the code being generated from an existing database. The main differences are:

Aspect	Code First	Database First
Source of truth	The C# entity classes	The existing database
Schema changes	Edit classes, then Add-Migration and Update-Database	Change the database manually, then re-scaffold the code
History and version control	Migration files are tracked in Git, giving a full history of schema changes	Database changes are not tracked in the codebase
Teams and environments	The same migrations rebuild an identical database on any machine	Needs a shared database or scripts, with a risk of drift
Control over code	Clean, hand-written entity classes	Generated classes that are overwritten on re-scaffold
Best suited for	New projects and domain-driven design	Existing or legacy databases

This matches the lab's rule that **C# code is the only source of truth**: no table is ever created or edited by hand in SQL Server. That said, Code First is not always the right choice - when a team must work with an existing legacy database owned by DBAs, Database First (scaffolding) is the practical option. For a new Bookstore system with no existing database, Code First is the natural fit.

2. Why do we use Fluent API over Data Annotations?

- **Clean entities:** Data Annotations put persistence details (attributes) directly inside the domain classes, coupling them to the mapping technology. With Fluent API the entities stay plain C# classes, and all database mapping lives in the configuration code.
- **More powerful:** Fluent API can express configuration that annotations cannot, such as composite keys (in EF Core 5), cascade-delete behavior, alternate keys, value conversions, global query filters, and seed data.
- **Highest precedence:** When configurations conflict, EF Core applies them in this order: conventions, then Data Annotations, then Fluent API - so Fluent API always has the final say.
- **Centralized and organized:** All mapping rules sit in one place (OnModelCreating), and with one configuration class per entity (EntityTypeConfiguration<T>) each entity's rules are easy to find, maintain, and test. This is exactly the "clean way" asked for in Part C.
- **No mixing of concerns:** Attributes like [Required] and [MaxLength] are also used for input validation (for example in ASP.NET MVC), so using them for mapping blurs the line between a validation rule and a database rule. Fluent API affects only the database model.

Trade-off: Data Annotations are quick and easy to read because they sit right next to the property, which is fine for simple rules in small projects. Many teams use annotations for the basics and Fluent API for everything else, but for larger, cleaner designs Fluent API is preferred.